

# MANUAL DE USUARIO

## Cómo se ejecuta:

El programa arranca con **pnpm run dev**, que corre **src/index.ts**. Este archivo solo llama a la función **menu()**, que muestra un menú en consola y queda esperando que el usuario elija una opción escribiendo un número.

## Menú principal:

Al iniciar, se listan 10 opciones divididas en dos bloques: productos (1 al 5) y clientes (6 al 10), más la opción 0 para salir. El usuario escribe el número de la opción y presiona Enter; el programa lee esa entrada con **readline** y ejecuta la acción correspondiente. Al terminar cada acción, el menú se vuelve a mostrar automáticamente (es recursivo), hasta que se elige la opción 0, que cierra la interfaz de lectura y termina el programa.

## Gestión de productos

- Agregar producto (1): pide código, nombre, precio y stock. Valida que el código y nombre no estén vacíos y que precio y stock sean números válidos y no negativos. Si el código ya existe, no lo agrega y avisa el error.
- Listar productos (2): lee el archivo **productos.json** y muestra todos los productos guardados.
- Buscar producto (3): pide un código y muestra el producto si existe, o un mensaje de "no encontrado".
- Actualizar producto (4): pide el código y los nuevos datos, valida que sigan siendo correctos, y si todo está bien reemplaza los valores y guarda el archivo.
- Eliminar producto (5): pide el código, lo busca y si existe lo quita del arreglo y guarda el archivo actualizado.

## Gestión de clientes

Funciona igual que productos, pero usando **id** (número) como identificador único en vez de código, y validando también que el correo tenga un "@" y que el teléfono no esté vacío. Las opciones son: agregar (6), listar (7), buscar (8), actualizar (9) y eliminar (10).

## Cómo se guarda la información

Todo se guarda en dos archivos JSON dentro de **src/data: productos.json** y **clientes.json**. Cada vez que se lista, busca, agrega, actualiza o elimina algo, el programa lee o escribe ese archivo completo usando funciones asíncronas basadas en

promesas (**readFile y writeFile de fs/promises**). Si el archivo no existe todavía, el programa no falla: simplemente empieza con una lista vacía.

### **Validaciones**

Antes de guardar cualquier producto o cliente, se revisan sus datos con una función de validación propia (**validarProducto o validarCliente**). Si algo no cumple las reglas (campo vacío, número negativo, correo sin "@", id duplicado, etc.), la operación no se realiza y se muestra el motivo del error.